

GOOGLE CAREER PREPARATION

DAY 8 — DETAILED TO-DO PLAN

Textbook-style study, implementation, testing, interview, and engineering checklist

Core Theme: Stacks + Queues + Backend Persistence Hardening + SQL Querying + Project Reliability

This document preserves the complete Day 8 plan from the preceding conversation, organized into a printable textbook-style format.

DAY 8 — DETAILED TO-DO PLAN

Core Theme

Day 8 = Stacks + Queues + Backend Persistence Hardening + SQL Querying + Project Reliability

Days 4-7 established:

Two Pointers



Sliding Window



Prefix Sum



Hashing



Intervals

Day 8 now introduces a new major DSA family:

STACKS

+

QUEUES

At the same time, Project 1 should move beyond merely "PostgreSQL works" toward:

API



Service



Repository



PostgreSQL



Reliable querying + validation + tests

This keeps your preparation aligned with the larger goal of becoming a Software Engineer first, specializing deeply in AI/ML/GenAI.

1. DAY 8 OBJECTIVES

By the end of Day 8 you should be able to:

DSA

- Recognize stack problems
- Recognize queue problems
- Explain LIFO vs FIFO
- Implement stacks in Python
- Implement queues efficiently
- Understand deque
- Use stacks for matching/nesting
- Use stacks for monotonic traversal
- Understand when a stack replaces recursion
- Understand queue-based BFS conceptually
- Solve 3 new DSA problems without reusing Days 4–7 problems

Backend

- Understand database query boundaries
- Understand filtering
- Understand ordering
- Understand pagination
- Understand why pagination matters
- Understand LIMIT/OFFSET
- Understand cursor/keyset pagination
- Understand parameterized SQL
- Understand SQL injection risks
- Understand repository responsibilities
- Understand service responsibilities
- Understand transaction boundaries

Project 1

Extend:

Document API



PostgreSQL

with:

Filtering

Ordering

Pagination

Validation

Repository-level query methods

Integration tests

2. DAY 8 TIME BUDGET

Target: ~3 hours

Area | Time

DSA theory | 30 min

DSA problems | 55 min

PostgreSQL/backend theory | 25 min

Project implementation | 55 min

Testing | 15 min

Interview + documentation | 20 min

Total | 3 hrs 20 min

If you only have exactly 3 hours, reduce documentation/interview time rather than removing DSA or project implementation.

3. DAY 8 — DSA

STACKS + QUEUES

3.1 First Learn the Mental Model

Stack

A stack follows:

LIFO — Last In, First Out

Example:

push(10)

push(20)

push(30)

30 ← top

20

10

pop() returns:

30

Then:

20

Then:

10

4. Stack Operations

Know:

push

pop

peek/top

is_empty

Typical complexity:

Operation | Complexity

Push | $O(1)$

Pop | $O(1)$

Peek | $O(1)$

Empty check | $O(1)$

5. Python Stack

Understand that a Python list can act as a stack:

```
stack = []
```

```
stack.append(value)
```

```
stack.pop()
```

```
stack[-1]
```

You should understand why:

```
stack.append()
```

```
stack.pop()
```

are appropriate stack operations.

Do not use:

```
pop(0)
```

as your stack operation.

6. Queue

A queue follows:

FIFO — First In, First Out

Example:

10 → 20 → 30

↑

front

Remove:

10

then:

20

then:

30

7. Why Python `list.pop(0)` Is a Problem

Understand:

`queue.pop(0)`

requires shifting remaining elements.

Therefore it is:

$O(n)$

per dequeue.

Instead study:

```
from collections import deque
```

```
queue = deque()
```

```
queue.append(value)
```

```
queue.popleft()
```

Both are designed for efficient queue operations.

8. Stack vs Queue

You must be able to explain:

Property | Stack | Queue

Ordering | LIFO | FIFO

Insert | Top | Rear

Remove | Top | Front

Python structure | list | deque

Common use | nesting/history | scheduling/BFS

9. Stack Pattern Recognition

When you see:

- matching parentheses
- nested structures
- undo operations
- previous/next unresolved element
- expression evaluation
- removing elements based on previous elements
- "nearest greater/smaller"
- need to process elements in reverse order

Think:

Could a stack represent the unresolved work?

This is more important than memorizing individual problems.

10. Queue Pattern Recognition

Think queue when you see:

- first-come-first-served
- level-order processing

- BFS

- scheduling
- shortest path in an unweighted graph
- process items in arrival order

11. MONOTONIC STACK — INTRODUCTION

This is the important Day 8 extension.

A monotonic stack maintains elements in:

increasing order

or:

decreasing order

Example:

[2, 1, 5, 3, 4]

The stack is used to efficiently determine:

What is the next greater/smaller element?

Instead of repeatedly scanning forward:

$O(n^2)$

we can often achieve:

$O(n)$

because each element is pushed and popped at most once.

12. DSA PROBLEM 1

Valid Parentheses

Do not treat this as merely an easy LeetCode problem.

Use it to learn the canonical stack pattern.

Example:

"{[()]}"

Process:

```
{  
[  
(  
)  
]  
}
```

The stack stores unmatched opening brackets.

When a closing bracket appears:

]

you inspect:

top = [

and verify that they match.

Learn

- push opening bracket
- pop on closing bracket
- detect mismatch
- detect leftover openings
- detect closing bracket with empty stack

Complexity

Time: $O(n)$

Space: $O(n)$

13. DSA PROBLEM 2

Daily Temperatures

This is your first serious monotonic stack problem.

Given:

[73,74,75,71,69,72,76,73]

find how many days you must wait until a warmer temperature.

Do not solve this using nested loops first and stop there.

You must understand:

monotonic decreasing stack

The stack stores indices whose warmer future day has not yet been found.

When:

`current_temperature > stack_top_temperature`

you resolve the previous unresolved day.

Key insight

The stack contains:

unresolved indices.

This concept will transfer to many Google-style problems.

Target

$O(n)$

rather than:

$O(n^2)$

14. DSA PROBLEM 3

Evaluate Reverse Polish Notation

Use this problem to understand:

Stack-based expression evaluation.

Example:

["2","1","+","3","*"]

Process:

2

1

+

→ 3

3

*

→ 9

You must understand:

- operands → push
- operator → pop operands
- calculate
- push result

Important

Understand operand ordering.

For:

$a - b$

the first popped value is:

b

and the second popped value is:

a

This is a common interview mistake.

15. DSA PROBLEM FORMAT

For all three problems:

Problem

↓

Pattern

↓

Recognition signal

↓

Brute force

↓

Why brute force is insufficient

↓

Optimized idea

↓

Invariant

↓

Algorithm

↓

Dry run

↓

Complexity

↓

Edge cases

↓

Common mistakes



Alternative approach



Interview follow-ups

For Daily Temperatures, explicitly document:

Why does each element enter the stack once?

Why does each element leave the stack once?

Why is total work $O(n)$?

16. DAY 8 DSA MINI-DRILL

After solving the three problems, spend 10 minutes without coding.

For each scenario identify:

Scenario A

"Find the matching closing bracket."

Answer:

Stack

Scenario B

"Process requests in arrival order."

Answer:

Queue

Scenario C

"Find the next greater element."

Answer:

Monotonic Stack

Scenario D

"Traverse a graph level by level."

Answer:

Queue + BFS

Scenario E

"Undo the most recent operation."

Answer:

Stack

The goal is pattern recognition, not memorization.

17. DAY 8 — BACKEND THEORY

Now move from:

PostgreSQL CRUD

to:

PostgreSQL querying

Your Day 5 work established PostgreSQL-backed CRUD and the principle that the API contract should remain stable while persistence changes behind the repository boundary.

Day 8 builds on that.

18. SQL FILTERING

Study:

SELECT *

FROM documents

WHERE document_type = 'requirement';

Then:

WHERE status = 'created'

Then multiple conditions:

WHERE document_type = 'requirement'

AND status = 'created';

Understand:

WHERE

filters rows before the result is returned.

19. ORDERING

Study:

ORDER BY created_at DESC;

and:

ORDER BY name ASC;

Understand:

ASC

DESC

Also understand why API consumers may need deterministic ordering.

For example:

```
GET /documents?sort=created_at&order=desc
```

20. PAGINATION

This is the main backend concept for Day 8.

Imagine:

1,000,000 documents

Returning all of them is not reasonable.

Instead:

```
GET /documents?limit=20&offset=0
```

Then:

```
GET /documents?limit=20&offset=20
```

Study:

LIMIT

OFFSET

Example:

SELECT ...

FROM documents
ORDER BY created_at DESC

LIMIT 20

OFFSET 40;

21. LIMIT/OFFSET TRADE-OFF

Understand the problem.

For small datasets:

LIMIT + OFFSET

is straightforward.

But at large offsets:

OFFSET 900000

can become increasingly expensive because the database may need to walk past many rows.

This leads to a later production concept:

Keyset/cursor pagination

Do not implement cursor pagination today unless you finish early.

Just understand why it exists.

22. PARAMETERIZED SQL

This is mandatory.

Never construct SQL like:

```
query = f"SELECT * FROM documents WHERE name = '{name}'"
```

Understand the SQL injection risk.

Instead use your database driver's parameterization mechanism.

Conceptually:

```
cursor.execute(  
"SELECT * FROM documents WHERE name = %s",  
    (name, ),
```

)

The exact API depends on your chosen PostgreSQL library.

The principle is:

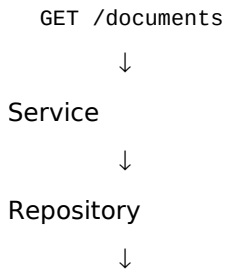
Data should be supplied as parameters, not concatenated into SQL strings.

23. PROJECT 1 — DAY 8

Objective

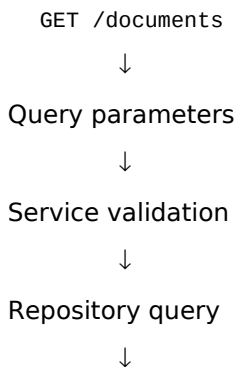
Add controlled document querying without breaking your existing architecture.

Current:



PostgreSQL

Day 8:



PostgreSQL

24. ADD PAGINATION

Extend:

GET /documents

with query parameters conceptually like:

limit

offset

For example:

```
GET /documents?limit=20&offset=0
```

and:

```
GET /documents?limit=20&offset=20
```

25. PAGINATION VALIDATION

Do not accept arbitrary values.

Define sensible constraints.

For example:

`limit >= 1`

`limit <= maximum_allowed`

`offset >= 0`

The exact maximum should be chosen deliberately and documented.

The important engineering concept is:

API input should be bounded before reaching the database.

26. ADD FILTERING

Add at least one meaningful filter based on your existing document model.

Potential example:

`document_type`

Conceptually:

```
GET /documents?document_type=requirement
```

Do not add five or ten filters.

One well-designed filter is enough for Day 8.

27. ADD ORDERING

Add controlled ordering.

For example:

```
GET /documents?sort=created_at&order=desc
```

Important security concept

Do not blindly insert arbitrary user input into:

ORDER BY

because column names generally cannot be safely handled the same way as ordinary values.

Instead use an allowlist.

For example conceptually:

ALLOWED_SORT_FIELDS = {

"created_at": "created_at",

"name": "name",

}

Then:

user input

↓

allowlist

↓

safe SQL column

This is a valuable backend interview concept.

28. REPOSITORY RESPONSIBILITY

The repository should handle:

SQL

database interaction

query construction

mapping database records

The service should handle:

business rules

validation

orchestration

The router should handle:

HTTP

request parameters

response

Therefore:

Router



Service



Repository

remains intact.

29. DO NOT PUT SQL IN THE ROUTER

Avoid:

```
@app.get("/documents")
def get_documents():
    cursor.execute(...)
```

The route should not become the database layer.

This would destroy the separation established earlier.

30. PROJECT TESTS

Add tests for the new behavior.

Test 1 — default pagination

```
GET /documents
```

Verify default behavior.

Test 2 — explicit limit

```
GET /documents?limit=2
```

Verify no more than two records are returned.

Test 3 — offset

Create several records.

Then:

```
GET /documents?limit=2&offset=2
```

Verify the expected page.

Test 4 — invalid limit

Example:

```
limit=0
```

Expected validation failure.

Test 5 — invalid offset

Example:

```
offset=-1
```

Expected validation failure.

Test 6 — filtering

```
GET /documents?document_type=requirement
```

Verify only matching documents.

Test 7 — ordering

Verify:

```
created_at DESC
```

actually produces deterministic ordering.

31. DATABASE TEST DATA

Make sure your tests deliberately create data that exposes ordering/filtering behavior.

Do not use:

document A

document B

with identical timestamps and assume ordering.

Create data that allows you to verify the actual sort.

This is an important testing habit:

Tests should make the expected behavior observable.

32. DAY 8 TRANSACTION CONCEPT

Do not build a complicated transaction system today.

Instead, understand where transactions belong.

Ask:

If a service operation requires three database writes, should they all succeed independently?

If the operation is logically atomic:

BEGIN

write A

write B

write C

COMMIT

If one fails:

ROLLBACK

You should understand this because the Project 1 architecture will eventually involve operations more complex than one-row CRUD.

33. DAY 8 BACKEND INTERVIEW QUESTIONS

You should answer these aloud.

Stack / Queue

- 1. What is a stack?**
- 2. What is a queue?**
- 3. What is LIFO?**
- 4. What is FIFO?**
- 5. Why is Python `list.pop(0)` inefficient?**
- 6. Why use `collections.deque`?**
- 7. What is a monotonic stack?**

- 8. Why can a monotonic stack achieve $O(n)$?**
- 9. How is a queue used in BFS?**
- 10. When would you use a stack instead of recursion?**

PostgreSQL

- 11. What is pagination?**
- 12. Why is pagination necessary?**
- 13. What is LIMIT?**
- 14. What is OFFSET?**
- 15. Why can large offsets become expensive?**
- 16. What is cursor/keyset pagination?**
- 17. What is SQL injection?**
- 18. How does parameterized SQL prevent injection?**
- 19. Why should arbitrary ORDER BY input not be directly concatenated?**
- 20. Where should SQL live in your architecture?**

Architecture

- 21. What belongs in the router?**
- 22. What belongs in the service?**
- 23. What belongs in the repository?**
- 24. Why shouldn't the service know SQL?**

25. Why shouldn't the router directly access PostgreSQL?

26. What happens if PostgreSQL is unavailable?

27. Where should transaction boundaries be defined?

28. How would you test pagination?

29. How would you test deterministic ordering?

30. How would you scale document listing to millions of records?

34. DAY 8 SYSTEM DESIGN MINI-EXERCISE

Spend 10-15 minutes on:

Design a document listing API capable of serving millions of documents.

Use your established system-design format:

Requirements



Scale



API



Data Model



Query Strategy



Indexes



Pagination



Caching



Consistency



Reliability



Monitoring



Cost

You should specifically discuss:

LIMIT/OFFSET

vs

cursor/keyset pagination

but do not implement the scaled architecture today.

35. DAY 8 DOCUMENTATION

Update your Project 1 documentation.

Add a section covering:

Document Querying

Document:

- pagination
- filters
- sorting
- validation
- query parameter behavior
- maximum page size
- ordering guarantees
- error responses

Also document the architectural boundary:

Router



Service



Repository



PostgreSQL

36. GOOGLE INTERVIEW PREP REPOSITORY

Create Day 8 material under the existing preparation repository.

Suggested structure:

day8/

```
■■■ dsa/
■   ■■■ stacks_notes.md
■   ■■■ queues_notes.md
■   ■■■ monotonic_stack_notes.md
■   ■■■ valid_parentheses.md
■   ■■■ daily_temperatures.md
■   ■■■ evaluate_reverse_polish_notation.md
■
■■■ backend/
■   ■■■ pagination_notes.md
■   ■■■ sql_injection_notes.md
■   ■■■ repository_query_design.md
■
■■■ system_design/
■   ■■■ document_listing_service.md
■
■■■ interview/
■   ■■■ day8_interview_questions.md
■
```

└─ README.md

Only create the files that you actually complete.

37. GIT COMMIT STRATEGY

Do not make one huge:

day8 completed

commit.

Prefer meaningful commits corresponding to actual completed work.

For example:

feat: add document pagination

feat: add document filtering

feat: add safe document ordering

test: cover document query parameters

docs: document document listing API

For DSA:

feat: add stack and queue dsa solutions

The actual commit messages should reflect what you genuinely changed.

38. DAY 8 — EXACT EXECUTION ORDER

00:00–00:10

Baseline

Run:

```
git status
git log --oneline -n 10
python -m pytest -q
```

Verify the repository is healthy before changing anything.

Output

Clean baseline.

00:10–00:40

Stack + Queue Theory

Study:

- stack
- queue

- LIFO

- FIFO

- Python list stack
- deque
- monotonic stack
- BFS relationship

Output

Explain each without notes.

00:40–01:35

DSA Problems

Solve:

1. Valid Parentheses

2. Daily Temperatures

3. Evaluate Reverse Polish Notation

No copying solutions.

For Daily Temperatures, explicitly derive the monotonic stack invariant.

01:35–02:00

Backend Theory

Study:

- filtering
- ordering
- pagination

- LIMIT/OFFSET

- cursor pagination concept
- parameterized SQL
- SQL injection
- repository boundaries

02:00–02:55

Project Implementation

Implement:

- pagination
- one filter
- controlled ordering
- validation
- repository query methods
- parameterized SQL
- allowlisted sort fields

Keep:

Router



Service



Repository



PostgreSQL

02:55–03:10

Testing

Run the full test suite.

Add tests for:

- default pagination
- limit
- offset
- invalid limit
- invalid offset
- filter
- ordering
- combined query parameters

03:10–03:20

Documentation + Git

Update:

- README

- API documentation
- DSA notes
- interview questions

Then commit and push.

39. DAY 8 DEFINITION OF DONE

Do not mark Day 8 complete because you solved three problems.

DSA

☐ Stack concept understood

- ☐ Queue concept understood
- ☐ LIFO/FIFO understood
- ☐ Python stack implemented
- ☐ deque understood
- ☐ Monotonic stack understood
- ☐ Valid Parentheses solved independently
- ☐ Daily Temperatures solved independently
- ☐ Evaluate Reverse Polish Notation solved independently
- ☐ Complexity explained
- ☐ Edge cases documented
- ☐ Pattern recognition recorded

Backend

- ☐ Filtering understood
- ☐ Ordering understood
- ☐ Pagination understood
- ☐ LIMIT understood
- ☐ OFFSET understood
- ☐ Cursor pagination understood conceptually
- ☐ SQL injection understood
- ☐ Parameterized SQL understood
- ☐ Repository/service/router boundaries understood

Project 1

- ☐ Pagination implemented
- ☐ Pagination validation implemented
- ☐ At least one filter implemented
- ☐ Ordering implemented
- ☐ Sort-field allowlist implemented
- ☐ Repository performs database queries
- ☐ SQL parameters are safely bound
- ☐ Router does not contain SQL
- ☐ Service remains responsible for business validation
- ☐ Existing CRUD behavior still works

Testing

- ☐ Existing tests still pass
- ☐ Pagination tested

- ☐ Limit tested
- ☐ Offset tested
- ☐ Invalid parameters tested
- ☐ Filter tested
- ☐ Ordering tested
- ☐ Combined query behavior tested
- ☐ No regressions

Documentation

- ☐ Query API documented
- ☐ Pagination documented
- ☐ Filtering documented
- ☐ Sorting documented
- ☐ Validation documented
- ☐ Architecture remains accurate
- ☐ DSA notes committed
- ☐ Interview questions committed

40. WHAT NOT TO DO ON DAY 8

Do not jump ahead to:

- Redis
- Docker
- Kubernetes

- GCP

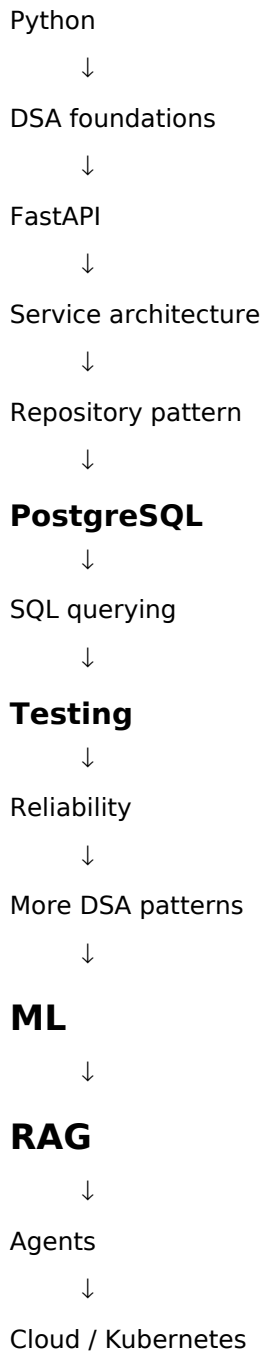
- React

- RAG

- embeddings
- vector databases
- agents
- LangChain
- MLflow
- model serving
- Kubernetes deployment
- microservices

You are deliberately building depth first.

The sequence should remain:



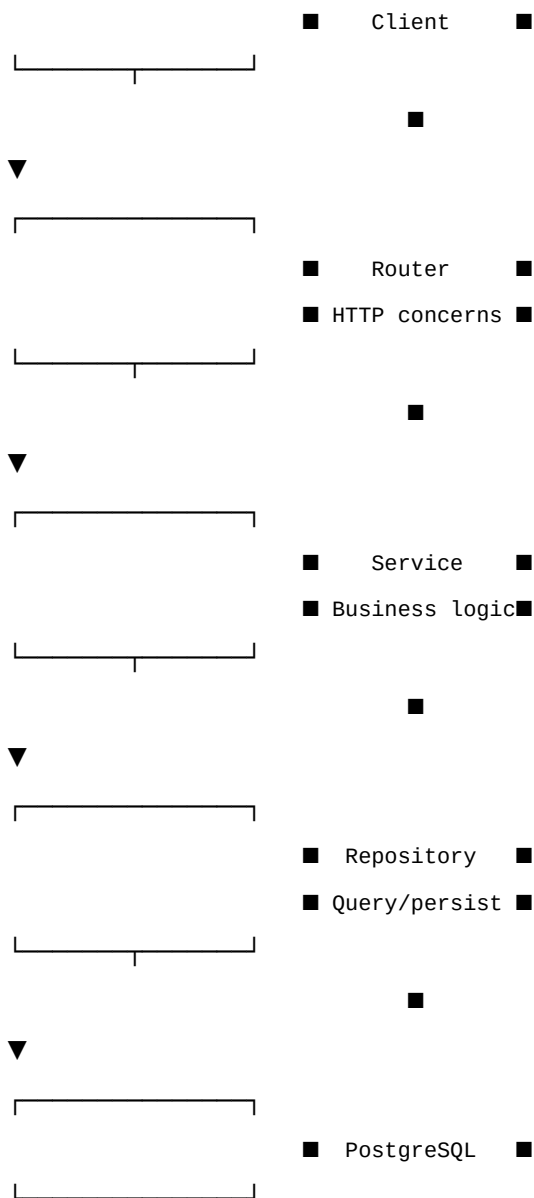
41. DAY 8'S MOST IMPORTANT ENGINEERING LESSON

The central lesson is not pagination.

It is this:

A backend system should separate HTTP concerns, business logic, and persistence/query logic.

You should be able to draw:



That architecture builds directly on the repository boundary established in the completed PostgreSQL work.

42. DAY 8 → DAY 9 DEPENDENCY

After Day 8, the DSA progression becomes:

Two Pointers



Sliding Window



Prefix Sum



Hashing



Intervals



Stack



Monotonic Stack



Queue



Trees / BFS / DFS

And Project 1 progresses:

CRUD



PostgreSQL



Testing



Filtering



Pagination



Ordering



Query abstraction



Next backend capability

Do not move to Redis or distributed infrastructure merely because PostgreSQL is working. The next step should be determined by what the repository actually contains after Day 8.

Day 8 success condition

By the end of the day, you should be able to explain, implement, test, and defend:

"How does a paginated, filtered document API travel from an HTTP request through FastAPI, business logic, a repository, parameterized PostgreSQL queries, and back to a validated HTTP response?"

while simultaneously being able to recognize:

"This is a stack problem," "this is a queue problem," or "this is a monotonic-stack problem."

That combination—DSA pattern recognition + production backend reasoning—is the core focus of Day 8.